

Chapter 1: From Autoencoders to VQ-VAE: Rethinking Visual Representation

1.1 The Dimensionality Problem in Image Generation

In the domain of machine learning and computer vision, there are generally two broad categories of tasks: discriminative modeling and generative modeling. For many years, the field was overwhelmingly dominated by discriminative models. These are systems designed to look at a highly complex piece of data—such as a high-resolution photograph of a dog—and reduce it down to a single, simple, discrete label: "Dog." This process is fundamentally an exercise in extreme data compression and pattern recognition. The network is taking millions of individual pixels and mapping them to a single concept. Because the network is actively discarding massive amounts of unnecessary information (such as the lighting in the room, the color of the background wall, or the specific angle of the camera), neural networks became incredibly proficient at this task very quickly.

Generative modeling, however, flips this paradigm on its head. It asks a neural network to do the exact opposite. Instead of taking an image and outputting a simple concept, we ask the network to start with a concept (or a sequence of random numbers) and output a full, high-resolution image from scratch.

This is an exponentially harder problem due to a mathematical phenomenon known as the "curse of dimensionality."

To understand why this is so difficult, consider a very small, low-resolution color image, measuring just 64 by 64 pixels. Because it is a standard color image, it has three color channels (Red, Green, and Blue) for every single pixel. To represent this tiny image, a computer needs $64 \times 64 \times 3$ numbers, which equals 12,288 individual values. If you want a generative neural network to imagine a completely new image of a human face, it must accurately predict all 12,288 numbers simultaneously.

If the network guesses these numbers randomly, the result will not be a face; it will be television static. To generate a coherent, recognizable image, the network must understand that these pixels are not statistically independent. They are highly correlated. If one pixel represents the color of human skin, the pixel directly next to it is highly likely to be a similar shade of skin. If there is a white cluster of pixels representing the sclera of an eye, there must be a dark pixel nearby representing the pupil.

To successfully generate an image, a machine learning model cannot simply memorize raw pixel values. It must understand global concepts: bilateral symmetry, lighting physics, 3D geometry, hair textures, and facial structures.

To teach a model these complex global concepts, we must first find a way to compress the massive, noisy grid of 12,288 pixels into a smaller, dense, and meaningful summary of the image's core features. In the field of machine learning, this compressed summary is called a "latent representation."

1.2 The Evolution of Image Compression: Autoencoders

The first logical attempt to compress an image into a latent representation is the standard Autoencoder (AE).

An Autoencoder is an unsupervised neural network architecture that consists of two distinct halves: an Encoder and a Decoder. These two halves are connected by a narrow central bottleneck, which forces the network to compress the data.

To understand this intuitively, imagine you have a highly detailed classical painting, and you need to describe it to a friend over the phone so they can recreate it on their end. You cannot describe every single brushstroke—it would take days, and the bandwidth of your phone call is limited. Instead, you act as the Encoder. You look at the painting and extract only the most vital, high-level information: "It is a portrait of a smiling woman, she has brown hair, and the background is a blurry green landscape." This short summary is the latent representation (often referred to mathematically as the latent space or the vector z).

Your friend on the other end of the phone acts as the Decoder. They take your short summary and use their own artistic knowledge to paint a reconstruction of the original image. Afterwards, you compare the original painting with your friend's reconstruction. You measure how mathematically different they are—this difference is called the Reconstruction Loss (most commonly measured by Mean Squared Error, or MSE).

During training, the neural network continuously adjusts its weights using backpropagation to minimize this reconstruction loss. Over thousands of iterations, the Encoder learns to write better, more mathematically efficient summaries, and the Decoder learns to interpret those summaries more accurately to rebuild the pixels.

This sounds like a perfect system. Autoencoders are indeed fantastic at data compression and removing noise from images. However, when researchers attempted to use standard Autoencoders as generative models to create brand new, never-before-seen images, they ran into a severe structural limitation: the interpolation problem.

The Interpolation Problem

When an Autoencoder trains, it only cares about successfully passing existing dataset images through its bottleneck. It maps every image it sees to a specific, arbitrary point in the latent space.

However, it does not organize this latent space in any meaningful, continuous way. It might place the compressed vector for a picture of a cat in the top-left corner of the latent space, and the vector for a picture of a dog in the bottom-right corner.

More importantly, the vast expanses of mathematical space between the cat and the dog are completely empty. The Decoder has never seen latent vectors from these empty spaces; it has only been trained on the exact coordinates of the real images. If you mathematically pick a random point halfway between the cat and the dog and ask the Decoder to generate an image from it, the Decoder panics. Because that specific coordinate in the latent space was never mapped to a real image during training, the Decoder outputs absolute garbage—usually a blurry, meaningless blob of noisy pixels.

Because the latent space of a standard Autoencoder is fundamentally disjointed, unstructured, and full of "dead zones," we cannot use it to generate new content. It acts as a strict memory bank, not a creative engine.

1.3 Bringing Order to Chaos: Variational Autoencoders (VAEs)

To solve the interpolation problem and make Autoencoders capable of generation, researchers introduced a brilliant mathematical upgrade: the Variational Autoencoder (VAE).

The primary goal of the VAE was to clean up the messy, disjointed latent space of the standard Autoencoder and force it to be smooth, continuous, and highly organized.

Instead of the Encoder looking at an image and outputting a single, exact point in the latent space (for example, the coordinate $[2.5, 3.1]$), the VAE's Encoder outputs a probability distribution. Specifically, it outputs a mean (a central location) and a variance (a radius of uncertainty around that location).

Returning to our painting analogy: instead of the Encoder telling your friend, "The person in the painting has exactly brown hair color #8B4513," the VAE Encoder says, "The person in the painting has hair that is somewhere in the brown distribution, give or take a little bit of shade variance."

During the forward pass of the network, the model randomly samples a point from

within this probability bubble. This is known as the "reparameterization trick." By forcing the Decoder to reconstruct the image from a slightly randomized point inside the bubble every single time, the Decoder is forced to learn robustness. It learns that points near each other in space should decode into very similar images.

KL Divergence and Latent Regularization

To prevent these probability bubbles from floating infinitely far apart (which would simply recreate the "dead zones" problem of the standard Autoencoder), the VAE introduces a strict mathematical rule called the KL Divergence penalty.

KL Divergence acts like a strong gravitational pull located at the exact center of the latent space (coordinate zero). It constantly pulls all the individual probability bubbles toward the origin, forcing them to overlap and merge into a single, giant, continuous, bell-shaped curve (a standard normal Gaussian distribution).

This solves the interpolation problem beautifully. Because the KL Divergence penalty forces all the latent variables to be packed tightly together with no empty spaces, you can now pick any random coordinate within that central blob, pass it to the Decoder, and get a realistic image. If you pick a point halfway between a cat and a dog, the overlapping probability distributions ensure the Decoder will generate a smooth, logical transition—a "cat-dog" hybrid.

Figure 1.1: Latent Space Topology: Standard Autoencoder vs. Variational Autoencoder

(Visual description: A side-by-side 2D plot comparison. The left plot shows scattered, hard dots with empty white space between them. The right plot shows translucent, overlapping circles or Gaussian bell curves centered around the origin (0,0).)

Caption: Comparison of latent spaces. Left: A standard Autoencoder maps images to isolated points, leaving undefined 'dead zones' that produce garbage outputs when sampled. Right: A Variational Autoencoder maps images to overlapping probability distributions, creating a continuous space suitable for interpolation.

For a long time, VAEs were the absolute gold standard for image generation. However, as researchers attempted to generate higher-resolution and more complex images, a devastating architectural flaw in the VAE was discovered.

1.4 The Trap of Posterior Collapse

The KL Divergence penalty is a double-edged sword. While it beautifully organizes and smooths the latent space, it acts as a harsh mathematical punishment during training. The neural network is constantly trying to minimize its overall loss, which in a VAE is the sum of the Reconstruction Loss (how good the image looks) and the KL Divergence penalty (how close the latent space is to a perfect Gaussian curve).

If we attempt to upgrade the VAE by giving it a very powerful, "smart" Decoder—specifically, an autoregressive decoder like a PixelCNN that predicts pixels sequentially by looking at the pixels it just generated—a phenomenon known as Posterior Collapse occurs.

Imagine a student taking a difficult exam. The student has a cheat sheet (the latent representation provided by the Encoder), but the teacher has instituted a strict rule: every time the student looks at the cheat sheet, they lose 5 points from their final grade (the KL Divergence penalty).

If the exam is very easy, or if the student is exceptionally smart, they will quickly realize that they can just guess the answers based on context clues without ever looking at the cheat sheet. By ignoring the cheat sheet entirely, they avoid the 5-point penalty and score a higher overall grade.

This is exactly what happens in a VAE paired with a powerful decoder. The decoder realizes that the latent vector z provided by the encoder is slightly noisy (due to the probability sampling) and costs a massive KL penalty to maintain. The decoder decides it is mathematically cheaper to simply ignore the latent vector entirely. Instead, it looks at the first pixel of the image, and uses its own internal logic to guess what the second pixel should be, then the third, and so on.

The encoder, realizing the decoder is ignoring its summaries, stops trying to encode any useful information. It collapses its output into pure, meaningless noise centered perfectly at zero to minimize the KL penalty as much as possible.

The latent space dies. The bottleneck becomes entirely useless. The network has successfully bypassed the entire architecture we built to compress the image.

Furthermore, even when posterior collapse is avoided, VAEs suffer from a secondary issue: inherent blurriness. Because VAEs rely on continuous probability distributions and Mean Squared Error, they tend to average out the sharp, high-frequency details of the real world. If the model is uncertain about whether a pixel should be black or white, it will output gray to minimize the squared error.

To generate truly sharp, coherent images without the risk of posterior collapse, the field needed a completely new way to think about latent representations.

1.5 The Paradigm Shift: Continuous vs. Discrete Representations

For decades, neural networks have been built on the fundamental assumption that representations must be continuous. We use floating-point numbers with infinite decimal places because they are differentiable, meaning we can smoothly calculate gradients and perform backpropagation to train the network.

But if we look at the real world, human reasoning and representation are rarely continuous. They are highly discrete.

Consider human language. Language is arguably the most powerful data compression algorithm in existence. If I want to describe an image to you, I do not provide a continuous array of light wavelengths and spatial frequencies. I use discrete words: "A brown dog sitting on green grass."

Words are categorical and discrete. A thing is either a "dog" or a "cat." There is no continuous, fractional state of being 0.62 dog and 0.38 cat in the human dictionary. By breaking the visual world down into a finite vocabulary of discrete concepts, we can describe incredibly complex scenes with immense efficiency and clarity.

If language is discrete, and language can describe images perfectly, why do we force our neural networks to represent images using infinite, continuous numbers?

What if, instead of asking an Autoencoder to output a continuous coordinate in space, we asked it to output a sequence of discrete visual "words"? What if we forced the network to build a dictionary of visual concepts, and restricted it to only communicating using the words available in that dictionary?

This philosophical realisation led to the creation of the Vector Quantised-Variational AutoEncoder (VQ-VAE).

Table 1.1: Summary of Autoencoder Architectures for Generative Modeling

Architecture	Latent Space Topology	Primary Loss Functions	Generative Limitation
Standard Autoencoder (AE)	Continuous (Deterministic)	Reconstruction (MSE)	Disjointed latent space prevents generation of new data and interpolation.
Variational Autoencoder (VAE)	Continuous (Probabilistic)	Reconstruction (MSE) + KL Divergence	Prone to Posterior Collapse; produces blurry images when paired with powerful decoders.
Vector Quantised-VAE (VQ-VAE)	Discrete (Categorical)	Reconstruction (MSE) + Codebook Loss + Commitment Loss	Hard to optimize natively due to the non-differentiable <code>argmax</code> quantization step.

1.6 Introducing the VQ-VAE (Vector Quantization)

The VQ-VAE proposes a radical shift in generative modeling: abandoning the continuous probability distributions of the VAE in favor of a rigid, discrete bottleneck.

To understand how this works, we must introduce the core component of the architecture: the Codebook.

Imagine a painter who is only allowed to use a specific, pre-defined palette of 512 distinct colors. They cannot mix the colors on the canvas to create new shades. They must look at the real world, choose the color from their palette that is the closest possible match to what they see, and apply it to the canvas.

In the VQ-VAE, the Codebook is exactly this palette. It is a dictionary containing a fixed number of vectors (for example, $K = 512$). Each of these 512 vectors represents a specific "visual word" or "visual patch" that the network can use to build an image.

Here is how the forward pass of a VQ-VAE operates conceptually:

1. The Encoder: An image is passed into the Convolutional Encoder. Just like a standard Autoencoder, the Encoder processes the image and outputs a grid of continuous vectors. Let's assume the output is an 8×8 grid, representing 64 different spatial regions of the image.
2. The Vector Quantizer (The Bottleneck): This is where the paradigm shifts. We take the first continuous vector from the 8×8 grid. We compare this vector against all 512 vectors sitting in our Codebook. We calculate the mathematical distance (usually L_2 distance) between them to find the

closest match.

3. The Index: Once we find the closest matching vector in the Codebook (let's say it is Codebook Vector #42), we throw away the continuous vector generated by the Encoder. We replace it entirely with Codebook Vector #42.
4. The Decoder: We repeat this nearest-neighbor search for all 64 regions of the image. What we are left with is an 8×8 grid of discrete integer indices (e.g., [42, 112, 7, 501...]). We take the actual codebook vectors corresponding to these specific indices and pass them to the Decoder. The Decoder reads these 64 visual words and attempts to reconstruct the original high-resolution image.

Figure 1.2: End-to-End Architecture of the VQ-VAE

(Visual description: A flowchart moving left to right: An Input Image $\rightarrow$ CNN Encoder block $\rightarrow$ an $8 \times 8 \times D$ continuous tensor block. A visual "nearest neighbor" (L_2 distance) operation comparing one cell of the tensor to a 512-slot 1D Codebook matrix. The tensor turns into an 8×8 grid of discrete integers (e.g., [42, 112, 7...]) $\rightarrow$ CNN Decoder block $\rightarrow$ Reconstructed Image.)

Caption: The forward pass of the VQ-VAE. The continuous output of the encoder is compared against a learned codebook. Each vector is replaced by the nearest codebook entry, creating a discrete grid of indices that is passed to the decoder.

This entire process is called Vector Quantization. We are taking a continuous, infinite space of vectors and forcing them (quantizing them) into a finite, discrete set of states.

Bypassing Posterior Collapse

Why is this architectural change such a massive breakthrough for generative modeling? Because it completely and elegantly solves the Posterior Collapse problem that plagued VAEs.

Remember that VAEs collapsed because the KL Divergence penalty actively punished the network for storing information in the latent space. In a VQ-VAE, there is no KL Divergence penalty.

Because the representation is discrete, we are no longer trying to force probability bubbles to overlap at the origin. There is no mathematical punishment for using the Codebook. The bottleneck is hard, absolute, and unavoidable. The Decoder cannot ignore the latent space, because the only information it receives are the discrete tokens chosen from the dictionary. By

removing the KL penalty and forcing a discrete bottleneck, the VQ-VAE ensures that the latent space remains highly active, rich, and deeply utilized.

1.7 The Mathematics of the Bottleneck: Solving the argmax Problem

While the theory of Vector Quantization is highly elegant, implementing it in a neural network introduces a catastrophic engineering nightmare.

Neural networks learn through a mathematical process called backpropagation. After an image passes forward through the network and a loss is calculated, the network must calculate the gradient (the slope) of the loss with respect to every weight in the network. This gradient tells the weights which direction to adjust themselves to lower the loss. Backpropagation relies entirely on the chain rule of calculus, which requires every operation in the network to be smooth and differentiable.

Vector Quantization relies on an argmax operation—specifically, "find the index of the closest codebook vector."

The argmax operation is not smooth. It acts like a hard, discrete staircase. If you shift the encoder's continuous output by a tiny amount, say 0.001, the closest codebook vector might remain exactly the same. Therefore, the local gradient is zero. However, if you shift it by another 0.001, it might suddenly cross a spatial boundary, and the index abruptly snaps from #42 to #43. The gradient at that exact microscopic boundary is mathematically infinite.

Because the gradient is either zero or infinity everywhere, calculus completely breaks down. The gradients cannot flow backward from the Decoder, through the Quantizer, to the Encoder. The Encoder would never receive any feedback on how to improve its continuous vectors, and the entire network would be untrainable.

The Straight-Through Estimator (STE)

To solve this, the authors of the original VQ-VAE paper introduced a brilliantly simple, almost audacious mathematical hack known as the Straight-Through Estimator.

The logic is beautifully pragmatic: if we cannot calculate the true gradient through the quantization step, we will just pretend the quantization step never happened during backpropagation.

During the forward pass of the network, we perform the hard Vector Quantization. We find the closest codebook vector, snap the representation to it, and pass it to the Decoder.

But during the backward pass, when the gradients are flowing from the Decoder back to the Encoder, we completely bypass the Quantizer. We take the gradients that the Decoder calculated for the codebook vectors, and we simply copy-paste them directly onto the continuous outputs of the Encoder.

Figure 1.3: The Straight-Through Estimator (STE) Gradient Bypass

(Visual description: A block diagram with two distinct paths. "Forward Pass" (Solid black arrows) flows left-to-right through an "Encoder", then a "Quantizer (argmax)", then to a "Decoder". "Backward Pass" (Dashed red arrows) flows right-to-left from the Decoder, mathematically loops around the Quantizer block (bypassing it entirely), and feeds directly into the Encoder.)

Caption: Because the argmax quantization step is not differentiable, the Straight-Through Estimator bypasses it during the backward pass. Gradients calculated at the decoder input are copied directly to the encoder output.

We are essentially telling the Encoder: "The Decoder wanted the chosen Codebook Vector to move in this specific direction to lower the loss. Since you are the one who chose that Codebook Vector, you should move your continuous output in that direction too."

Despite being a blatant mathematical approximation, the Straight-Through Estimator works exceptionally well in practice, allowing the gradients to flow smoothly and enabling the entire VQ-VAE to train end-to-end flawlessly.

1.8 Codebook Learning and the Three-Part Loss Function

We have solved how to pass gradients to the Encoder and the Decoder, but we are missing one final, crucial component: how do we train the Codebook itself?

When the network is first initialized, the 512 vectors inside the Codebook are filled with random noise. If they remain random noise, the network will never learn to represent anything meaningful. The dictionary words must physically adapt to the data. If the dataset consists of human faces, the Codebook vectors must move through the mathematical space to position themselves as optimal representations of "eyes," "skin," and "hair."

Because of the Straight-Through Estimator bypass, the Codebook vectors themselves do not receive gradients from the standard reconstruction process. Therefore, to train all three components (Encoder, Decoder, Codebook) simultaneously, the VQ-VAE relies on a unique three-part loss function.

1. The Reconstruction Loss This is the standard loss used in all autoencoders. It measures the mathematical difference between the original input image and the final image produced by the Decoder. This loss provides the gradients that teach the Encoder how to extract better features, and teaches the Decoder how to turn those features back into realistic pixels.

2. The Codebook Loss (Dictionary Learning) Because the Straight-Through Estimator bypasses the dictionary, we need a separate mathematical rule to train the codebook vectors.

The Codebook Loss essentially performs an online version of K-Means clustering. It measures the L_2 distance between the continuous vector produced by the Encoder and the chosen discrete vector from the Codebook. The loss acts as a rubber band, pulling the static Codebook vector closer to the continuous Encoder output.

Over thousands of training steps, this loss systematically drags the randomly initialized Codebook vectors towards the dense clusters of data being produced by the Encoder, turning them into highly accurate, specialized representations of the dataset's visual features.

3. The Commitment Loss While the Codebook Loss pulls the dictionary words toward the Encoder, we also have to worry about the Encoder behaving erratically.

The volume of the latent space is essentially infinite. If the Encoder's weights update too aggressively, its continuous outputs could swing wildly from one end of the space to the other. If an Encoder's output for a specific image patch jumps from Codebook Vector #10 to Codebook Vector #450 between training batches, the Decoder will be completely confused, and the network will destabilize.

To prevent this, the Commitment Loss is added. This loss does the exact inverse of the Codebook loss: it penalizes the Encoder for straying too far from the Codebook vector it just selected. It forces the Encoder's continuous outputs to stay closely clustered around the dictionary words, ensuring that the Encoder "commits" to its choices. This maintains a stable, predictable representation for the Decoder to rely on.

Figure 1.4: Dictionary Learning and Latent Commitment Dynamics

(Visual description: A simple 2D coordinate plane with two plotted points: A blue circle (labeled "Encoder Output z_e ") and a red square (labeled "Codebook Vector e_k "). A red arrow points from the square to the circle (labeled "Codebook Loss"). A blue arrow points from the circle to the square (labeled "Commitment Loss").)

Caption: Visual representation of the VQ-VAE alignment losses. The Codebook Loss pulls the dictionary vector toward the encoder's output, while the Commitment Loss constrains the encoder's output from drifting too far from the chosen dictionary vector.

(Note: While the original paper proposes this direct loss calculation for the codebook, modern implementations—including the one developed for this project—often use an Exponential Moving Average (EMA) to update the codebook vectors, which yields much higher stability. This engineering decision will be explored in depth in Chapter 2).

1.9 Conclusion: The Power of Discrete Tokens

By combining these three loss components, the Straight-Through Estimator, and a fixed Codebook bottleneck, the VQ-VAE achieves something remarkable. It takes the messy, unconstrained latent space of a standard Autoencoder, strips away the destructive KL Divergence penalty of the Variational Autoencoder, and produces a highly stable, deeply compressed representation of visual data.

More importantly, it fundamentally changes the nature of the data itself. By pushing a 64 \times 64 color image through a trained VQ-VAE, we compress it into a tiny 8 \times 8 grid of discrete integers.

We have successfully translated the continuous, high-dimensional chaos of raw pixels into a flat, sequential array of discrete tokens—just like words in a sentence.

Figure 1.5: Treating Images as Sequences of Discrete Tokens

(Visual description: An image of a face is shown passing through the VQ-VAE encoder, resulting in an 8x8 grid of numbers. An arrow shows this grid being flattened into a 1D sequence (e.g., [SOS, 42, 112, 7, 501, 18...]), resembling a sentence of text.)

Caption: By flattening the 2D grid of discrete latent indices into a 1D array, the image is transformed into a sequence of tokens, functionally identical to a sentence of words in natural language processing.

Once visual data is formatted like language, we unlock an incredible new capability. We can borrow the most powerful generative architectures from Natural Language Processing—such as the causal Transformer—and teach them to "read" and "write" these visual tokens.

Because the VQ-VAE handles the heavy lifting of translating pixels into tokens, the Transformer does not need to worry about the physics of lighting or the noise of individual pixels. It only needs to learn the logical sequence of the visual words. By training an autoregressive language model on the discrete outputs of a VQ-VAE, we can teach an AI to generate entirely new, high-quality images, token by token, from scratch.

In the next chapter, we will transition from theory to practice. We will explore the actual system architecture and engineering required to build this exact end-to-end pipeline from the ground up, utilizing multi-GPU processing, custom convolutional networks, Exponential Moving Averages for stable codebook learning, and a causal Mini-GPT to autoregressively imagine human faces.

Chapter 2: System Architecture and Engineering the Training Pipeline

2.1 Why VQ-VAE Was Chosen For This Project

The ultimate goal of this engineering project was not simply to reconstruct images. Autoencoders have been capable of image reconstruction for over a decade. Instead, the primary objective was to build an end-to-end autoregressive generation system capable of "imagining" entirely new human faces from scratch.

To achieve this, we needed an architecture that could bridge the gap between continuous visual data (pixels) and sequence-based generative models (Transformers). Transformers, like GPT, are incredibly powerful at modeling sequences, but they require discrete inputs—a finite vocabulary of "words" or "tokens."

Standard Variational Autoencoders (VAEs) output continuous probability distributions, making them fundamentally incompatible with the discrete, token-by-token processing of a Transformer. Furthermore, as discussed in the previous chapter, VAEs suffer from posterior collapse when paired with powerful autoregressive models, causing them to ignore the compressed latent space entirely.

The Vector Quantised-Variational AutoEncoder (VQ-VAE) was the natural engineering choice for this project. It solves both problems simultaneously. By forcing the image data through a rigid, discrete codebook bottleneck, it completely bypasses the posterior collapse problem. More importantly, it acts as a translator. It converts the high-dimensional, continuous chaos of raw pixels into a neat, 1D sequence of discrete integer tokens. Once the VQ-VAE translates the images into a language-like vocabulary, a Transformer can be trained to model the statistical relationships between those tokens, allowing for the generation of coherent, high-quality images.

The following table summarizes the core components of the implemented VQ-VAE system and their specific engineering purposes:

Component	Purpose
Encoder	Compress image
Codebook	Store visual vocabulary
Quantizer	Convert continuous vectors to discrete tokens
Decoder	Reconstruct image
STE	Enable gradient flow
Commitment Loss	Stabilize encoder
Codebook Loss	Learn dictionary

2.2 Dataset Preparation and Design Decisions

The foundation of any machine learning pipeline is the data. For this project, the CelebFaces Attributes (CelebA) dataset was selected. CelebA is a massive, publicly available dataset containing over 200,000 images of celebrity faces.

Human faces are widely considered an ideal benchmark for generative modeling. Faces possess a strict global geometric structure—the eyes, nose, and mouth must be aligned in very specific relative positions for the image to look human. However, faces also exhibit extremely high local variance in the form of different lighting conditions, skin tones, hair textures, and background environments. This makes CelebA a rigorous test: the model must learn both

strict global rules and diverse local textures.

Preprocessing Decisions

Raw images from the CelebA dataset vary in framing and contain a significant amount of background clutter. To force the network to focus strictly on facial features, a CenterCrop operation of 178 pixels was applied. This successfully removed the chaotic background elements and centered the subjects.

Next, the images were resized to a resolution of 64×64 pixels. This was a critical engineering tradeoff. Generating high-resolution images (e.g., 256×256) is visually impressive but exponentially increases the computational load and memory requirements, particularly for the Transformer phase later in the pipeline. A resolution of 64×64 is small enough to allow for rapid iteration and training on consumer-grade hardware, yet large enough to retain distinct, recognizable facial features.

Finally, the pixel values were normalized. Standard image formats store pixels as integers ranging from 0 to 255. Feeding these raw values into a neural network causes unstable gradients and extremely slow convergence. The pixels were first converted to floating-point tensors between $[0, 1]$ and then mathematically shifted to a range of $[-1, 1]$.

This specific $[-1, 1]$ range was explicitly chosen to match the Tanh (Hyperbolic Tangent) activation function used at the very end of our custom Decoder network. By aligning the dataset's numerical bounds with the network's output bounds, the model is theoretically capable of perfectly reproducing the data scale without requiring additional unscaled linear layers.

Before allowing the training pipeline to begin, automated shape verification and sanity checks were engineered into the data loader. The system extracts a dummy batch of 128 images, verifying that the tensor shape is exactly $[128, 3, 64, 64]$ and that the minimum and maximum values strictly adhere to the $[-1, 1]$ boundaries. This defensive programming step prevents silent mathematical failures during deep training runs.

2.3 Designing the VQ-VAE Architecture

With the data pipeline established, the next phase was designing the custom Convolutional Neural Network (CNN) architecture for the Encoder and Decoder.

The Encoder and Spatial Compression

The Encoder's job is to compress the spatial dimensions of the image while

increasing the feature depth. Instead of using MaxPooling layers—which deterministically discard data and can create harsh artifacts—the architecture utilizes strided convolutions. A convolution with a stride of 2 halves the spatial resolution of the feature map while allowing the network to learn exactly how to downsample the image optimally.

The input image (64×64) passes through three sequential blocks of strided convolutions. The first block reduces the image to 32×32 , the second to 16×16 , and the final block to an 8×8 grid. Following best practices for encoder networks, LeakyReLU activation functions were used to prevent "dead neurons" during early training, and Batch Normalization was applied after every convolution to keep internal representations mathematically stable.

The 8×8 Latent Grid Tradeoff

The decision to compress the image specifically to an 8×8 spatial grid was one of the most important engineering choices in the project.

The final stage of this project involves training a Transformer. Transformers possess an attention mechanism that scales quadratically with the sequence length ($O(N^2)$). An 8×8 spatial grid flattens into a 1D sequence of exactly 64 tokens. A sequence length of 64 is highly computationally efficient for a Transformer.

If we had chosen a 16×16 grid, the sequence length would balloon to 256 tokens. The Transformer would require 16 times more memory and compute to process the exact same image. Conversely, if we compressed the image down to a 4×4 grid (a sequence length of 16), the compression ratio would be too extreme. An entire human face cannot be accurately represented by only 16 patches, and the resulting reconstructions would be unacceptably blurry. The 8×8 grid provided the optimal tradeoff between spatial fidelity and Transformer efficiency.

Sizing the Codebook and Embeddings

The Quantizer bottleneck required two distinct hyperparameters: the size of the Codebook (how many visual words exist) and the embedding dimension (the mathematical size of each word).

The embedding dimension was set to 256. This means that every single cell in the 8×8 grid is represented by an array of 256 floating-point numbers. This matches the output channel depth of the final Encoder layer, providing enough continuous capacity for the CNN to cluster complex textures without wasting GPU

memory.

The Codebook size was set to 512. Choosing the dictionary size is a delicate balancing act. If the dictionary is too small (e.g., 64 words), the model lacks the vocabulary to describe complex scenes, resulting in severe compression artifacts and loss of detail. If the dictionary is unnecessarily large (e.g., 8192 words), a phenomenon called "Codebook Collapse" occurs. The model simply will not use the vast majority of the words, leaving thousands of vectors completely dead. Furthermore, a smaller codebook makes the subsequent Transformer's job much easier, as it only has to predict probabilities across 512 possible tokens rather than thousands.

The Decoder

The Decoder acts as an exact structural mirror of the Encoder. It takes the 8×8 grid of 256-dimensional codebook vectors and passes them through three blocks of Transposed Convolutions (often informally called deconvolutions). These layers systematically upsample the spatial grid from 8×8 to 16×16 , then 32×32 , and finally back to the original 64×64 resolution. The final layer outputs exactly 3 channels (RGB) and passes through a Tanh activation function to guarantee the output values are bounded perfectly between $[-1, 1]$, matching the normalized input dataset.

2.4 Engineering the Quantization Layer

The Vector Quantizer is the architectural bridge between the Encoder and Decoder. Translating the theoretical concept of nearest-neighbor lookup into fast, highly optimized GPU code presented a unique engineering challenge.

In a naive implementation, finding the closest codebook vector would involve writing a for loop to compare the Encoder's output against every single one of the 512 vectors in the Codebook. In Python, explicit for loops are notoriously slow. Running this loop for every pixel patch across a batch of 128 images would cause training times to stretch from hours into weeks.

To solve this, the distance computation was engineered using a vectorized matrix expansion. The Euclidean distance formula $(a - b)^2$ can be algebraically expanded into $a^2 + b^2 - 2ab$.

By leveraging PyTorch's highly optimized matrix multiplication routines, we calculated the sum of the squares of the Encoder outputs (a^2), added the sum of the squares of the Codebook weights (b^2), and subtracted twice the dot product of the two matrices ($2ab$). This mathematical trick computes the exact distance between every single Encoder output and every single Codebook vector

simultaneously in one massive, parallelized GPU operation.

Once the distances are computed as a large matrix, we simply apply an `argmin` operation across the codebook dimension. This instantly returns the discrete integer indices (from 0 to 511) of the closest matching codebook vectors.

Finally, these indices are passed into a standard PyTorch `nn.Embedding` layer. This layer acts as a simple lookup table, retrieving the actual 256-dimensional continuous vectors corresponding to those indices. The tensor is then carefully reshaped and permuted back into the [Batch, Channels, Height, Width] format required by the convolutional Decoder. At this exact moment in the forward pass, continuous representations are successfully destroyed and replaced by discrete visual tokens.

2.5 Solving the Training Challenges

Because the `argmin` function used to select the closest codebook index is a discrete step function, its mathematical gradient is zero almost everywhere. If implemented natively, the backpropagation chain would sever at the Quantizer, the Encoder would receive no gradients, and the network would be entirely untrainable.

The Straight-Through Estimator Implementation

To bridge this severed gradient path, the Straight-Through Estimator (STE) was implemented using a cleverly engineered tensor operation.

In the code, the quantized vector is redefined as: `quantized = encoder_output + (quantized - encoder_output).detach()`

To understand why this is brilliant, we must look at how PyTorch handles the `.detach()` command. The `.detach()` command tells the automatic differentiation engine to completely ignore whatever is inside the parentheses during the backward pass.

During the forward pass, the equation mathematically evaluates to: `quantized = encoder_output + quantized - encoder_output`. The two `encoder_output` terms cancel each other out perfectly. The forward pass behaves exactly as expected, passing the discrete quantized tensor to the Decoder.

During the backward pass, however, PyTorch ignores the detached section. It looks at the equation and only sees: `quantized = encoder_output`. Therefore, whatever gradient the Decoder calculates for the quantized tensor is copied 1:1 directly onto the `encoder_output`. The mathematical roadblock is bypassed.

entirely, and the Encoder learns how to improve.

The Flaw in the Original Loss Formulation

With the gradient path restored, the network can minimize the Reconstruction Loss (Mean Squared Error). To ensure this loss remained numerically stable regardless of the contrast or brightness of the specific batch of images, the MSE was explicitly divided by the mathematical variance of the dataset. This keeps the scale of the reconstruction loss consistent and prevents it from overpowering the other loss metrics.

We also implemented the Commitment Loss, which applies a small penalty if the Encoder's continuous outputs drift too far away from the Codebook vectors they selected.

However, a major engineering roadblock was encountered regarding the Codebook itself. The original VQ-VAE paper updates the Codebook vectors using a direct loss metric (Codebook Loss) optimized via standard gradient descent (e.g., the Adam optimizer).

In practice, this is highly unstable. Adam uses momentum, meaning it accelerates updates based on past gradients. Furthermore, training batches are inherently noisy. If a specific Codebook vector (e.g., Vector #104, representing a specific shade of blonde hair) happens to not be used by any images in a particular batch, it receives no gradients. Meanwhile, the surrounding vectors are being violently shifted by Adam's momentum. Vector #104 gets left behind in empty space. Because it is now far away from the data clusters, it is never chosen again in future batches. It becomes a "dead code." Over a few epochs, a large percentage of the 512 vectors can die, leading to severe Codebook Collapse.

2.6 Stabilizing the Codebook with EMA

To prevent dead codes and ensure a healthy, highly utilized Codebook, the standard gradient descent update for the dictionary was entirely ripped out and replaced with an Exponential Moving Average (EMA) update strategy.

EMA treats the codebook update not as a neural network loss optimization, but as an online version of the classic K-Means clustering algorithm.

Instead of letting an optimizer push the codebook vectors around blindly, the EMA system maintains two highly specific running trackers, updated locally during the forward pass:

1. Cluster Size Tracker: A tally of exactly how many times each of the 512

codebook vectors was chosen in the current batch.

2. Vector Sum Tracker: The mathematical sum of all the continuous Encoder vectors that were mapped to a specific codebook vector.

If we simply divided the Vector Sum by the Cluster Size, we would find the exact geometric centroid (the average center) of all the data points that mapped to that codebook vector.

Instead of jumping directly to that centroid, we use an Exponential Moving Average with a decay factor (typically 0.99). The system takes the Codebook vector's previous location, keeps 99% of it, and shifts it 1% of the way toward the newly calculated centroid.

Laplace Smoothing and Dead Code Prevention

A critical edge case must be engineered around: what happens if a Codebook vector is chosen exactly 0 times in a batch? The Cluster Size is zero. Dividing the Vector Sum by zero would cause a NaN (Not a Number) explosion, instantly crashing the entire training pipeline.

To solve this, Laplace smoothing was applied to the tracking mechanism. A tiny numerical epsilon ($1e-5$) was added to the denominators across the board. This completely prevents division-by-zero crashes. Furthermore, the cluster sizes are artificially normalized to ensure they always sum to the total number of codebook vectors, maintaining strict mathematical boundaries.

Because EMA updates are smooth, deliberate, and independent of optimizer momentum, a code that isn't used in one batch doesn't get violently thrown out of position. It simply stays where it is, retaining its historical relevance. This EMA engineering decision was the single most impactful choice in the VQ-VAE pipeline, resulting in near 100% codebook utilization across all 512 tokens during our final training runs.

2.7 System Engineering and Infrastructure

Training generative models requires aggressive memory management and hardware optimization. A standard implementation using 32-bit floating-point precision (fp32) would quickly exhaust the Video RAM (VRAM) of standard GPUs, forcing the use of tiny batch sizes and massively slowing down convergence.

Mixed Precision Training

To accelerate training, Automatic Mixed Precision (AMP) was implemented using `torch.autocast`. This system automatically analyzes the mathematical operations

happening inside the network. Operations that require high precision (like distance calculations and loss accumulation) are kept in 32-bit floats. However, operations that are highly parallel and structurally forgiving (like standard Convolutional layer matrix multiplications) are dynamically downcast to 16-bit floats (fp16). This effectively halves the memory footprint of the heavy layers and utilizes specialized GPU Tensor Cores, yielding massive speedups.

However, fp16 introduces a new engineering hazard: gradient underflow. Because 16-bit floats cannot represent incredibly small numbers, tiny gradients can round down to exactly 0.0, causing the network to stop learning. To prevent this, a GradScaler was introduced. Before the backward pass begins, the scaler multiplies the final loss by a large factor (e.g., 65536). This artificially inflates the gradients so they safely survive the 16-bit math. After backpropagation is complete, the scaler divides the gradients back down to their true size before applying the optimizer step.

Multi-GPU Architecture and Checkpointing

To further accelerate the system, the training loop was engineered to support multiple GPUs using PyTorch's `nn.DataParallel`. This wrapper automatically splits a single massive batch of 128 images into smaller micro-batches, distributes them across available GPUs, computes the forward and backward passes in parallel, and gathers the gradients back to the master node.

While highly effective, `DataParallel` introduces a notorious engineering quirk. It wraps the core VQ-VAE model inside a `.module` container. When attempting to access custom layers—such as the Codebook matrix to plot the usage histogram—or when saving the model checkpoints to the hard drive, calling the model directly fails. The pipeline was explicitly engineered with safety checks to detect if multiple GPUs are present and safely extract the `base_model = model.module` before saving weights to disk or performing latent extractions.

Combating Zombie Memory

A practical challenge encountered during the iterative development of this pipeline was "zombie memory" leaks. When running training loops repeatedly in an interactive notebook environment, interrupted runs often leave massive graphical tensors stranded in the GPU memory. Starting a new run would immediately trigger an Out Of Memory (OOM) crash.

To ensure the training loop was bulletproof, explicit Python garbage collection (`gc.collect()`) and CUDA cache clearing (`torch.cuda.empty_cache()`) commands were engineered into the very top of the training execution block, guaranteeing a clean memory slate before allocating heavy tensors.

2.8 Building the Latent Dataset

Once the VQ-VAE training completed successfully, the model had learned a robust, 512-word discrete visual vocabulary. The first phase of the project was officially over. The next engineering challenge was bridging the gap between the VQ-VAE and the Transformer.

To do this, we had to permanently convert the entire CelebA dataset from continuous pixels into discrete sequences.

The trained VQ-VAE was placed into evaluation mode (`model.eval()`) and its gradient tracking was completely disabled (`torch.no_grad()`). This freezes the network weights and dramatically reduces memory consumption, as PyTorch no longer needs to store computational graphs.

We then passed the entire dataset of over 200,000 images through the frozen Encoder and Vector Quantizer. For every single 64×64 image, the VQ-VAE output an 8×8 grid of discrete integer indices.

Because Transformers expect 1D sequences, not 2D grids, these 8×8 grids were mathematically flattened into flat arrays of 64 tokens.

Storing this massive new dataset required careful data engineering. The default data type for integers in NumPy and PyTorch is `int64` (which uses 64 bits per number). Storing 200,000 sequences of length 64 using `int64` wastes an immense amount of RAM and disk space. Because our maximum vocabulary size is 512, we cast the entire extracted dataset to `uint16` (unsigned 16-bit integers, which can store values up to 65,535). This simple engineering optimization reduced the dataset's memory footprint by 75%, allowing it to be comfortably saved as a lightweight NumPy array (`celeba_8x8_latents.npy`).

At this exact moment, a fundamental conceptual shift occurred in the project. The images were no longer images. They were officially tokenized sequences. They were sentences.

2.9 Designing the Mini-GPT Transformer

With the dataset transformed into sequences of visual words, we engineered the final component of the pipeline: an autoregressive Transformer model designed to learn the statistical grammar of human faces.

Because we are generating tokens sequentially—predicting the next token based entirely on the previous tokens—we utilized a decoder-only causal Transformer,

architecturally identical to the foundation of modern GPT models.

Vocabulary and Embeddings

The first design decision was the vocabulary size. While the VQ-VAE Codebook has exactly 512 visual tokens, the Transformer's vocabulary size was explicitly set to 513.

The 513th token (Index 512) serves as the crucial Start-Of-Sequence (SOS) token. When generating an image from scratch, the Transformer needs a prompt to begin. The SOS token acts as this initial seed, telling the network, "Begin generating a face."

When a sequence of integers enters the Mini-GPT, the integers themselves carry no mathematical meaning (Token #42 is not inherently "greater" than Token #12). They must be converted into continuous vectors. The network uses a `nn.Embedding` layer to map each of the 513 possible indices into a rich, 256-dimensional Token Embedding space.

Positional Embeddings and Causal Masking

Transformers process entire sequences simultaneously; they have no inherent concept of order or geometry. If we simply fed the visual tokens into the network, it would not know if a token belonged to the top-left corner of the face or the bottom-right corner.

To solve this, Positional Embeddings were engineered into the input layer. We generate a sequence of position indices (0 to 63) and map them to their own 256-dimensional embeddings. The Token Embeddings and the Positional Embeddings are mathematically summed together. This ensures that the Transformer knows exactly what the visual patch is, and exactly where it belongs in the spatial grid.

To make the model autoregressive, a Causal Mask was applied to the core Attention mechanism. A causal mask is essentially a triangular matrix of negative infinities. It forces the self-attention mechanism to be strictly unidirectional. When the network is evaluating Token #10, the mask physically prevents it from "looking ahead" at Token #11. It can only draw context from Tokens 0 through 10. This strict constraint forces the network to learn genuine predictive patterns rather than just memorizing the sequence.

Architecture and Objective

The core Transformer block was sized to balance generative capability with

hardware constraints. It consists of 6 sequential layers, each featuring 8 multi-head attention mechanisms and a feed-forward dimension of 1024. This "Mini-GPT" configuration contains roughly several million parameters—large enough to learn complex spatial dependencies across the 64-token sequence, but small enough to train efficiently.

The final layer of the network is a linear projection mapping the 256-dimensional hidden state back up to 513 logits. The network is trained using a standard Cross-Entropy objective. Exactly like a language model trying to predict the next word in a sentence, the Mini-GPT looks at a sequence of visual tokens and outputs a probability distribution predicting the exact integer index of the next visual patch. To ensure training stability and prevent the "exploding gradient" problem common in deep Transformers, explicit gradient clipping (`torch.nn.utils.clip_grad_norm_`) was engineered into the optimization loop, capping the maximum gradient magnitude at 1.0.

2.10 Chapter Summary

Through careful architectural design and systematic hardware optimizations, the continuous problem of image generation was successfully broken down into a discrete sequence-modeling task.

By engineering a strided convolutional Encoder, implementing the Straight-Through Estimator to restore gradient flow, and leveraging Exponential Moving Averages to stabilize Codebook learning, the VQ-VAE successfully translated raw pixels into a highly optimized vocabulary of 512 visual words. We then extracted this vocabulary, flattened it into a highly efficient sequence dataset, and engineered a causal Mini-GPT Transformer to learn the autoregressive grammar of those visual tokens.

The pipeline is fully constructed. The models are capable of communicating. In the next and final chapter, we will analyze the empirical results of this system, interpret the training dynamics and loss curves, and evaluate the final generated faces to understand both the triumphs and the limitations of discrete visual representation.

Chapter 3: Experimental Analysis, Results, Challenges, Limitations and Future Directions

3.1 Evaluating the VQ-VAE Training Process

The first major milestone of this project was successfully training the Vector Quantised-Variational AutoEncoder (VQ-VAE). To evaluate whether the system was learning effectively, we continuously monitored two primary metrics during the training loop: the Reconstruction Loss and the VQ Codebook Loss.

During our final training run, the Normalized Mean Squared Error (MSE) Reconstruction Loss steadily decayed, ultimately stabilizing at approximately 0.0829.

A decaying reconstruction loss indicates that the Encoder is successfully learning to map raw pixels into continuous vectors, and the Decoder is successfully learning to translate the discrete codebook tokens back into a recognizable image. Because we normalized the dataset variance out of the loss calculation, a value of 0.0829 represents a highly accurate structural reconstruction relative to the data's overall contrast.

However, in the context of a VQ-VAE, a low reconstruction loss alone does not guarantee a useful latent representation. If we only looked at the reconstruction loss, we might assume the model is perfect. But a neural network is an optimization machine; it will find the mathematically easiest way to lower its loss. If the bottleneck is poorly designed, the network might rely on a tiny handful of codebook vectors to reconstruct a blurry, generic face, completely ignoring the rest of the dictionary.

To verify that the model was genuinely learning a rich vocabulary, we analyzed the VQ Loss curve. The VQ Loss measures the distance between the continuous vectors produced by the Encoder and the discrete vectors sitting in the Codebook. In a typical successful training run, this loss does not smoothly decay to zero. Instead, it often rises initially or fluctuates before stabilizing. This behavior occurs because the Codebook vectors are initializing, moving, and spreading out across the high-dimensional space to accurately cluster the diverse facial features provided by the Encoder.

Figure 3.1: Reconstruction Loss Curve

(Visual description: A line graph showing the Normalized MSE dropping sharply during the first few epochs and smoothly tapering off to a horizontal asymptote near 0.0829.)

Caption: The reconstruction loss over training epochs. The steady decay indicates the convolutional autoencoder successfully learned to compress and decompress the spatial geometry of human faces.

Figure 3.2: VQ Loss Curve

(Visual description: A line graph showing the Codebook alignment loss rising initially before stabilizing at a plateau, indicating the codebook vectors have spread out to map the latent space.)

Caption: The VQ Codebook loss during training. The stabilization of this metric confirms that the dictionary vectors have found optimal clustering positions within the continuous encoder space.

3.2 Analyzing Codebook Utilization

The most critical health metric of any VQ-VAE system is its Codebook Utilization—exactly how many of the available discrete tokens the network is actively using to build images.

As discussed in Chapter 2, "Codebook Collapse" is a catastrophic failure mode where the network abandons the vast majority of its vocabulary and relies on only a few "safe" tokens. Dead codes occur when a vector is initialized in a mathematically empty region of the latent space. If standard gradient descent is used, the momentum of the optimizer pushes the active vectors around, while the unused vectors are left behind, permanently ignored by the nearest-neighbor lookup.

To combat this, we implemented an Exponential Moving Average (EMA) update strategy with Laplace smoothing. The goal was to mimic online K-Means clustering, updating vectors based solely on their historical usage rather than erratic gradient momentum.

The results of this engineering decision were overwhelmingly successful. Upon analyzing the final trained model, the system achieved a perfect utilization rate: 512 out of 512 active codebook entries.

This result is highly significant. It proves that the continuous mathematical space generated by the Encoder was uniformly distributed and that the EMA tracking successfully dragged every single initialized codebook vector into a meaningful cluster of data. A 100% utilization rate means that the subsequent Transformer model will have a rich, highly descriptive vocabulary of 512 distinct visual patches to work with, rather than a crippled dictionary of only 20 or 30 generic textures.

Figure 3.3: Codebook Usage Histogram

(Visual description: A bar chart with the x-axis ranging from 0 to 511 (the codebook indices) and the y-axis showing the frequency of use. Every single bar is populated, showing a relatively uniform distribution with no empty gaps.)

Caption: Codebook usage across the final validation batch. The lack of zero-value columns demonstrates that the EMA update strategy completely prevented dead codes, achieving 100% vocabulary utilization.

Table 3.1: Codebook Utilization Statistics		
Metric	Value	Interpretation
Total Codebook Size	512	The maximum allowed vocabulary of visual tokens.
Active Vectors	512	Every token was used to encode the dataset.
Utilization Rate	100%	Total avoidance of Codebook Collapse.
Update Strategy	EMA + Laplace	Provided the numerical stability necessary for perfect utilization.

3.3 Visual Quality of Reconstructions

Before attempting to generate new images from scratch, we must evaluate the VQ-VAE's ability to reconstruct existing images. This isolates the performance of the compression bottleneck. If the VQ-VAE cannot reconstruct a recognizable face from an 8 \times 8 grid of tokens, the Transformer will have no hope of generating one.

When comparing the original 64 \times 64 CelebA images against their VQ-VAE reconstructions, the system's ability to preserve global information is immediately apparent.

What was preserved: The model successfully preserved the fundamental structural geometry of the subjects. The orientation of the face, the positioning of the eyes, the alignment of the jawline, and the general hairstyle were translated flawlessly through the 64-token bottleneck. Furthermore, global color palettes—such as skin tone, hair color, and the general hue of the background—were highly accurate. The network clearly learned specific tokens for "dark brown hair patch," "light skin patch," and "blue background patch."

What was lost: As expected from a highly compressed representation, high-frequency local details were lost. Individual strands of hair merged into smooth, cohesive shapes. Sharp transitions between the subject and the background were slightly softened. Subtle textures, like freckles or sharp specular highlights in the eyes, were averaged out.

Why it happened: This loss of detail is a direct consequence of the aggressive spatial compression. We forced a 64×64 image into an 8×8 grid. This means every single discrete token is responsible for representing an 8×8 pixel patch of the original image. It is mathematically impossible to store the exact arrangement of 64 individual pixels inside a single vocabulary word. The VQ-VAE is forced to act as an impressionist painter, capturing the "essence" of the patch rather than the exact photographic detail.

Figure 3.4: Original vs Reconstructed Images

(Visual description: A side-by-side grid of faces. The top row shows the original, sharp CelebA faces. The bottom row shows the VQ-VAE reconstructions, which maintain the exact identities, colors, and poses, but appear slightly softer, resembling high-quality digital paintings.)

Caption: Reconstructions from the VQ-VAE. While the global structure, pose, and color palettes are accurately preserved, the strict 8×8 token bottleneck forces the model to average out high-frequency textures like individual hair strands.

3.4 Training and Evaluating the Mini-GPT

With the visual vocabulary verified, the project moved to the sequence modeling phase. The entire dataset was translated into sequences of length 64, and the causal Mini-GPT Transformer was trained to predict the next token in the sequence.

The primary metric for evaluating the Transformer was the Cross-Entropy Loss. During training, this loss steadily converged to approximately 3.14.

To understand what a loss of 3.14 actually means, we must consider the mathematical baseline. The Transformer is trying to guess the next visual token out of a vocabulary of 512 possible tokens. If the network were completely untrained and guessing randomly, the probability of guessing correctly would be $\frac{1}{512}$. The Cross-Entropy loss for random guessing is the negative natural logarithm of this probability: $-\ln(1/512) \approx 6.24$.

A loss of 3.14 is drastically lower than the random baseline of 6.24. We can reverse the math to understand the network's confidence: $\exp(3.14) \approx 23$. This means that at any given step in the 64-token sequence, the Transformer has successfully narrowed down the 512 possible choices to a highly probable subset of roughly 23 logical tokens.

This proves that the Transformer has successfully learned the "visual grammar" of human faces. It learned spatial dependencies: if it generates a "left eye" token at position 18, it knows it absolutely must not generate another "left eye" token at position 19. It knows that hair tokens belong near the edges of the sequence, and nose and mouth tokens belong in the center. By combining the causal masking (which prevents looking ahead) with positional embeddings (which tell the model where it is on the grid), the Transformer built a deep statistical understanding of facial geometry.

Figure 3.5: Transformer Training Curve

(Visual description: A line graph showing the Cross-Entropy loss starting high and smoothly converging to a plateau just above 3.0.)

Caption: The Mini-GPT training curve. The convergence to a Cross-Entropy loss of ~ 3.14 demonstrates that the model learned strong predictive correlations between the discrete visual tokens, performing significantly better than random chance.

3.5 End-to-End Autoregressive Generation

The ultimate test of the system is end-to-end autoregressive generation—imagining a face that does not exist in the dataset.

The inference pipeline operated exactly as designed. The Transformer was seeded with a single Start-Of-Sequence (SOS) token. It predicted the probability distribution for the first pixel patch.

To select the token, we utilized Temperature Sampling combined with multinomial selection. If we simply always picked the single most likely token (argmax), the model would generate the exact same, generic, "average" face every time. By converting the logits into probabilities and sampling from them, we introduce controlled randomness, allowing the model to generate a vast diversity of different faces.

This process was repeated in a loop 64 times. Crucially, the probability of selecting the SOS token again was artificially forced to zero, ensuring the model didn't trap itself in a restart loop. Once the sequence of 64 tokens was generated, it was reshaped into an 8×8 grid, passed through the Vector Quantizer's lookup table, and fed into the VQ-VAE Decoder to render the final RGB pixels.

The results were a definitive success. The system successfully generated coherent, AI-imagined human faces from scratch.

The generated faces exhibit strict geometric coherence. The eyes are symmetric and level. Noses are positioned correctly between the eyes and the mouth. Backgrounds are separated logically from the foreground subjects. This proves that an entirely language-based architecture—predicting discrete integers one by one—is fully capable of understanding and rendering complex 2D spatial structures.

Figure 3.6: Generated Face Samples

(Visual description: A grid of entirely novel, AI-generated human faces. The faces show a variety of genders, skin tones, and hair colors, all possessing correct facial geometry.)

Caption: Autoregressively generated faces. The causal Transformer generated these images one discrete token at a time. The correct placement of facial features proves the model successfully learned the global spatial structure of a human face.

3.6 Failure Analysis and Limitations

While the structural coherence of the generated images is a triumph, an honest engineering analysis requires a thorough examination of the system's failures. The most glaring limitation of the generated output is its overall blurriness. The imagined faces lack the crisp, photorealistic sharpness associated with modern state-of-the-art AI.

This blurriness is not a bug; it is a direct mathematical consequence of two specific design choices: the Compression Bottleneck and the Mean Squared Error (MSE) objective function.

The Pixel Averaging Effect of MSE: During the training of the VQ-VAE, the Decoder is punished by MSE. MSE calculates the squared difference between the generated pixel color and the real pixel color.

Imagine the network is trying to reconstruct a patch of hair. Hair is full of high-frequency details—sharp, alternating lines of dark brown and light brown. If the Decoder tries to guess exactly where a specific dark hair strand is, but places it one pixel too far to the left, the MSE penalty is massive because it placed a dark pixel where a light pixel was supposed to be.

The neural network quickly learns that trying to guess high-frequency, sharp details is mathematically dangerous. Instead, the safest way to minimize MSE is to output a smooth, average blur of medium brown. The model is essentially hedging its bets. It sacrifices sharpness to guarantee a low statistical error.

The Limitation of Transformer Capacity: Furthermore, the resolution was limited to 64×64 . To generate a sharp 256×256 image using this exact architecture, the 8×8 latent grid would need to expand to 32×32 . This creates a sequence of 1,024 tokens. Because the Attention mechanism inside the Mini-GPT scales quadratically ($O(N^2)$), processing a 1,024-token sequence requires exponentially more GPU memory and compute than a 64-token sequence. The architecture is structurally bottlenecked by the context window limit of the Transformer.

The model easily learned the global structure (the macroscopic placement of the eyes and mouth) because global structure is represented by the sequence of the tokens. However, it failed to learn fine textures because fine texture must be stored inside the individual continuous embeddings, which are heavily penalized by MSE blurriness.

Figure 3.7: Examples of Lost Fine Details

(Visual description: A zoomed-in comparison showing the sharp, distinct strands of hair in a real CelebA image next to the smooth, blended, painterly hair of a generated image.)

Caption: The pixel-averaging effect of Mean Squared Error. Because MSE heavily penalizes slight spatial misalignments of sharp edges, the network learns to output a "safe," blurry average, resulting in a loss of high-frequency textures like hair strands.

3.7 Engineering Challenges and Lessons Learned

Building this pipeline from scratch provided profound practical engineering insights that theoretical papers rarely discuss.

Codebook Stability is Fragile: The most significant lesson learned was that discrete optimization is inherently unstable. Our early attempts to use standard optimizers (Adam) to update the Codebook resulted in immediate Codebook Collapse. The realization that dictionary learning should be treated as an online K-Means clustering problem—utilizing Exponential Moving Averages (EMA) and Laplace smoothing—was the turning point of the project. It highlighted that mathematical elegance in a paper does not always translate to stable GPU code.

Memory Management in PyTorch: Deep learning is an exercise in resource management. When implementing the causal Transformer, we frequently encountered Out Of Memory (OOM) errors. We learned to aggressively utilize `torch.autocast` for mixed-precision (fp16) training, which effectively doubled our batch sizes.

However, fp16 introduced gradient underflow, requiring the implementation of a GradScaler.

Furthermore, "zombie memory" in interactive environments caused repeated crashes. We engineered explicit garbage collection (`gc.collect()`) and CUDA cache clearing into the training loops to guarantee a clean memory slate.

The Quirks of DataParallel: Scaling the training across multiple GPUs using `nn.DataParallel` was necessary for speed, but introduced severe friction when trying to save models or access internal variables. `DataParallel` wraps the model in a `.module` container, breaking direct references to the `vq_layer.embedding`. We learned to write defensive code that dynamically checks for the presence of `DataParallel` and strips the wrapper before attempting to save checkpoints or extract the discrete dataset.

These challenges reinforced the idea that machine learning engineering is not just about designing architectures; it is equally about managing state, memory, and hardware limitations.

3.8 Future Directions

The architecture built in this project represents the foundational breakthrough that made modern generative AI possible. However, the field has evolved rapidly to solve the exact limitations (blurriness and resolution constraints) that we experienced.

If this project were to be scaled further, several state-of-the-art upgrades would be implemented:

Solving Blurriness with VQGAN: To fix the "safe average" blurriness caused by MSE, modern systems replace MSE with a Perceptual Loss (LPIPS) and a PatchGAN discriminator. This upgrades the VQ-VAE into a VQGAN. The discriminator looks at the generated image and penalizes it if it looks "fake" or "blurry," forcing the decoder to take risks and generate sharp, high-frequency textures like pores and hair strands, even if they aren't perfectly aligned with the original image.

Solving the Resolution Limit with Hierarchical Latents (VQ-VAE-2): To generate higher resolution images without blowing up the Transformer's sequence length, architectures like VQ-VAE-2 use a hierarchical structure. They compress an image into a "top-level" highly compressed grid (to capture global structure) and a "bottom-level" less-compressed grid (to capture fine details). The Transformer predicts the global structure first, then uses that as a map to predict the fine details.

Scaling to Text-to-Image (DALL·E): The original OpenAI DALL·E uses the exact architecture built in this project. The only difference is that they prepended a sequence of text tokens to the sequence of visual tokens. By training a massive Transformer on [Text Tokens] + [Visual Tokens], the model learned to conditionally generate images based on text descriptions.

The Shift to Latent Diffusion: While autoregressive Transformers are powerful, generating images one token at a time is computationally slow. Modern systems like Stable Diffusion replace the autoregressive Transformer entirely. They still use a continuous Autoencoder to compress the image, but instead of predicting discrete tokens in a sequence, they use a U-Net to gradually denoise the continuous latent space.

Table 3.2: Evolution Beyond VQ-VAE		
Technology	Limitation Solved	How it Improves Generation
VQGAN	MSE Blurriness	Uses adversarial loss to force the decoder to generate sharp, photorealistic textures.
VQ-VAE-2	Sequence Length Bottlenecks	Uses hierarchical codebooks to compress global and local features separately, enabling high-resolution scaling.
DALL·E	Lack of Control	Concatenates text tokens with discrete visual tokens, allowing text-prompted generation.
Latent Diffusion	Slow Autoregressive Generation	Replaces sequence generation with a parallelized denoising process in a continuous latent space.

|

3.9 Conclusion

This project successfully engineered an end-to-end generative pipeline that bridges the gap between spatial computer vision and sequential natural language processing.

By implementing a Vector Quantised-Variational AutoEncoder from scratch, we bypassed the historical limitations of continuous latent spaces. Through careful engineering of Exponential Moving Averages, we prevented Codebook Collapse and successfully compressed high-dimensional human faces into a highly optimized, discrete vocabulary of 512 visual words. We then successfully trained a causal Mini-GPT Transformer to learn the statistical grammar of these words, culminating in the autoregressive generation of coherent, novel human faces.

While the generated images exhibit the blurriness inherent to MSE reconstruction and extreme compression, the architectural integrity of the system holds strong.

The model successfully learned global facial geometry purely from observing sequences of integers.

This project demonstrates a profound truth in modern machine learning: if you can translate your data into a discrete, sequential language, a Transformer can learn to speak it. By approaching image generation not as a problem of painting pixels, but as a problem of writing sentences, we unlocked the foundational mechanics that power the modern era of generative artificial intelligence.